Measuring Data Similarity and Dissimilarity

Similarity and Dissimilarity

- **Similarity**
  - Numerical measure of how alike two data objects are
  - Value is higher when objects are more alike
  - Often falls in the range [0,1]
- **Dissimilarity** (e.g., distance)
  - Numerical measure of how different two data objects are
  - Lower when objects are more alike
  - Minimum dissimilarity is often 0
  - Upper limit varies
- **Proximity** refers to a similarity or dissimilarity

Data Matrix and Dissimilarity Matrix

- Data matrix
  - n data points with p dimensions

- Dissimilarity matrix
  - n data points, but registers only the distance
  - A triangular matrix

Proximity Measure for Nominal Attributes

- Can take 2 or more states, e.g., red, yellow, blue, green (generalization of a binary attribute)
- Method 1: Simple matching
  - m: # of matches, p: total # of variables

Object / Attribute	Color	Quality
i	red	good
j	blue	good

$$d(i,j) = \frac{2-1}{2} = \frac{1}{2} = 0.5$$

Proximity Measure for Nominal Attributes

Method 2: Use a large number of binary attributes

- creating a new binary attribute for each of the *m* nominal states
- Let, Color = {red, yellow, blue, green} and
  Quality = {good, bad}

Object	red	yellow	blue	green	good	bad
i	1	0	0	0	1	0
j	0	0	1	0	1	0

- For asymmetric:

$$d(i,j) = \frac{1+1}{6} = \frac{2}{6} = 0.33$$

$$d(i,j) = \frac{1+1}{3} = \frac{2}{3} = 0.66$$

Proximity Measure for Binary Attributes

- A contingency table for binary data

		Object j	
		1	0
Object i	1	q	r
	0	s	t
	sum	$q+s$	$r+t$

- Distance measure for symmetric binary variables:

- Dista $d(i,j) = \frac{r+s}{q+r+s+t}$ ry variables:

$$d(i,j) = \frac{r+s}{}$$

Jaccard Coefficient

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

- The Jaccard Similarity will be 0 if the two sets don't share any values and 1 if the two sets are identical.
- The set may contain either numerical values or strings.
- Additionally, this function can be used to find the dissimilarity between two sets by calculating d(A,B)=1–J(A,B)
- **Example:** Compute the Jaccard similarity:
- A={0,1,2,5,6} and B={0,2,3,4,5,7,9}
- Jaccard Similarity between two sets is calculated as follows:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|\{0,2,5\}|}{|\{0,1,2,3,4,5,6,7,9\}|} = \frac{3}{9} = 0.33$$

Jaccard Coefficient for Binary Attributes

- A contingency table for binary data

		Object j	
		1	0
Object i	1	q	r
	0	s	t
	sum	$q+s$	$r+t$

- Jaccard coefficient (*similarity* measure for *asymmetric* binary variables):

$$sim_{Jaccard}(i,j) = \frac{q}{q+r+s}$$

- Note: Jaccard coefficient is the same as "coherence":

$$coherence(i,j) = \frac{sup(i,j)}{sup(i)+sup(j)-sup(i,j)} = \frac{q}{(q+r)+(q+s)-q}$$

Dissimilarity using Asymmetric Binary Variables

- Example:
  - Gender is a symmetric attribute (nominal)
  - The remaining attributes are asymmetric binary
  - Let the values Y and P be 1, and the value N be 0

Dissimilarity using Interval-Scaled Variables

- Commonly used methods are distance based dissimilarity.
- Generally, variables are standarized before measuring dissimilarity.
- Standarization Steps:

1. Calculate the mean absolute deviation, Sf , where x_1f, …, xnf are n measurements of f, and mf is the mean value of f, i.e.,

$$m_f = \frac{1}{n}(x_{1f} + x_{2f} + \cdots + x_{nf})$$

$$S_f = \frac{1}{n}(|x_{1f} - m_f| + |x_{2f} - m_f| + \ldots + |x_{nf} - m_f|)$$

2. Calculate the standardized measurement, or Z-score:

$$z_{if} = \frac{x_{if} - m_f}{S_f}$$

Example:
Data Matrix and Dissimilaritv Matrix

Data Matrix

Dissimilarity Matrix
(with Euclidean Distance)

Distance on Numeric Data: Minkowski Distance

- *Minkowski distance*: A popular distance measure

$$d(i, j) = \sqrt[h]{|x_{i1} - x_{j1}|^h + |x_{i2} - x_{j2}|}$$

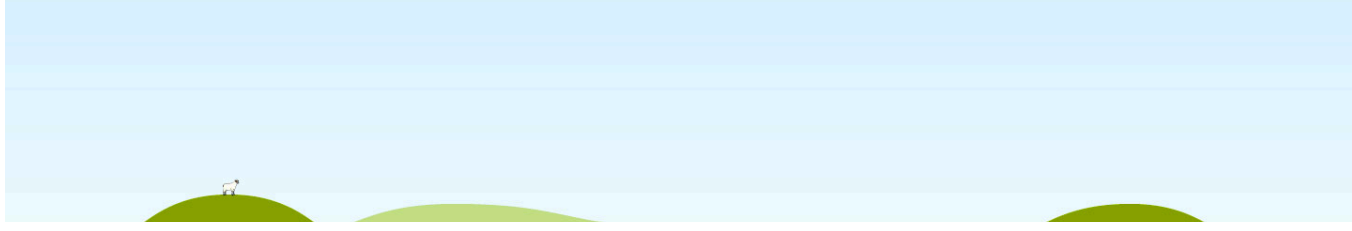
- where $i = (x_{i1}, x_{i2}, \ldots, x_{ip})$ and $j = (x_{j1}, x_{j2}, \ldots, x_{jp})$ are two p-dimensional data objects, and h is the order (the distance so defined is also called L-h norm)
- Properties
  - d(i, j) > 0 if i ≠ j, and d(i, i) = 0 (Positive definiteness)
  - d(i, j) = d(j, i) (Symmetry)
  - d(i, j) d(i, k) + d(k, j) (Triangle Inequality)
- A distance that satisfies these properties is a metric

Special Cases of Minkowski Distance

- *h* = 1: Manhattan (city block, L_1 norm) distance
  - E.g., the Hamming distance: the number of bits that are different between two binary vectors

- *h* = 2: (L_2 norm) Euclidean distance

- *h* . "supremum" (Lmax norm, L norm) distance.
  - This is the maximum difference between any component (attribute) of the vectors



Example: Minkowski Distance

Dissimilarity Matrices

Manhattan (L_1)

Euclidean (L_2)

Supremum

Ordinal Variables

- An ordinal variable can be discrete or continuous
- Order is important, e.g., rank
- Can be treated like interval-scaled
  - replace x_{if} by their rank
  - map the range of each variable onto [0, 1] by replacing i-th object in the f-th variable by Min-Max normalization
  - compute the dissimilarity using methods for interval-scaled variables (like Euclidean distance etc.)

Ratio-Scaled Variable

- A ratio scaled variable makes a positive measurement on a nonlinear scale, such as an exponential scale, approximately following the formula AeBt or Ae-Bt, where A and B are positive constants.
- Typical examples: growth of a bacteria population, decay of a radioactive element
- How to compute dissimilarity: Generally 3 methods:
  1. Treat ratio-scaled variables as interval scaled variables. This is not usually a good choice since it is likely that the scaled may be distorted.

Ratio-Scaled Variable

2. Apply logarithmic transformation to a ratio-scaled variable f having value xif for object i by using the formula yif = log(xif), and then yif values can be treated as interval-valued. For some ratio-scaled variables, log-log or other transformation may be applied, depending on the definition and application.
3. Treat xif as continuous ordinal data and treat their ranks as interval-valued.

- The latter two methods are the most effective, although the choice of method used may be dependent on the given application.

Attributes of Mixed Type

- A database may contain all attribute types
  - Nominal, symmetric binary, asymmetric binary, numeric, ordinal
- One may use a weighted formula to combine their effects

Where the indicator $\delta ij^{(f)} = 0$ if if either

- xif or xjf is missing or
- xif = xjf =0 and variable f is asymmetric binary

Otherwise: $\delta ij^{(f)} = 1$

Attributes of Mixed Type

- The contribution of variable f to the dissimilarity between i and j, i.e., $dij^{(f)}$, is computed depending on its type:
  - If f is binary or nominal:
    $dij^{(f)} = 0$ if $xif = xjf$, otherwise $dij^{(f)} = 1$
  - If f is interval-scaled: , where
    h runs over all nonm$d_{ij}^{(f)} = \frac{|x_{if} - x_{jf}|}{max_h x_{hf} - min_h x_{hf}}$e f.
  - If f is ordinal or ratio-scaled:
    - Compute ranks rif and
    - Treat zif as interval-scaled

Cosine Similarity

- A **document** can be represented by thousands of attributes, each recording the *frequency* of a particular word (such as keywords) or phrase in the document.

Document	*team*	*coach*	*hockey*	*baseball*	*soccer*	*penalty*	*score*	*win*	*l*
Document1	5	0	3	0	2	0	0	2	
Document2	3	0	2	0	1	1	0	1	
Document3	0	7	0	2	1	0	0	3	
Document4	0	1	0	0	1	2	2	0	

- Applications: information retrieval, biologic taxonomy, gene feature mapping, ...
- Cosine measure: If d_1 and d_2 are two vectors (e.g., term-frequency vectors), then

  $\cos(d_1, d_2) = (d_1 \cdot d_2) / \|d_1\| \|d_2\|$,

  where $\cdot$ indicates vector dot product, $\|d\|$: the length of vector d

Example: Cosine Similarity

- $\cos(d_1, d_2) = (d_1 \cdot d_2) / \|d_1\| \|d_2\|$,
  where $\cdot$ indicates vector dot product, $\|d\|$: the length of vector d

- Ex: Find the **similarity** between documents 1 and 2.

  $d_1 = (5, 0, 3, 0, 2, 0, 0, 2, 0, 0)$
  $d_2 = (3, 0, 2, 0, 1, 1, 0, 1, 0, 1)$

  $d_1 \cdot d_2 = 5*3+0*0+3*2+0*0+2*1+0*1+0*0+2*1+0*0+0*1 = 25$
  $\|d_1\| = (5*5+0*0+3*3+0*0+2*2+0*0+0*0+2*2+0*0+0*0)^{0.5} = (42)^{0.5} = 6.481$
  $\|d_2\| = (3*3+0*0+2*2+0*0+1*1+1*1+0*0+1*1+0*0+1*1)^{0.5} = (17)^{0.5} = 4.12$
  $\cos(d_1, d_2) = 0.94$

Criteria	Cosine Similarity	Euclidean Similarity
Definition	Measures the cosine of the angle between two vectors.	Measures the distance between two vectors.
Magnitude Sensitivity	Insensitive to the magnitude of vectors.	Sensitive to the magnitude of vectors.
Dimensionality	Well-suited for high-dimensional spaces.	Works better in lower-dimensional spaces.
Data Sparsity	Effective for sparse data, such as text data.	May not perform well with sparse data.
Orthogonality	Handles orthogonality well, distinguishing between similar vectors even if their magnitudes vary.	Tends to struggle with orthogonality, as it considers the direct distance.
Normalization	Requires normalized vectors for accurate results.	Normalization is not required.
Application	Commonly used in natural language processing, document similarity, and recommendation systems.	Commonly used in clustering, classification, and dimensionality reduction.
Computation Complexity	Generally less computationally intensive.	Can be computationally intensive in high dimensions due to the square root operation.

Feature Selection

Feature Selection

- Recent advancements in IT have produced the rapid production of big data, as enormous volumes of data with high dimensional features grow exponentially in different fields.
- Therefore, dealing with high-dimensional data creates new challenges in terms of data processing efficiency and effectiveness.
- To address such challenges, Feature Selection (FS) is among the most utilized dimensionality reduction methods.

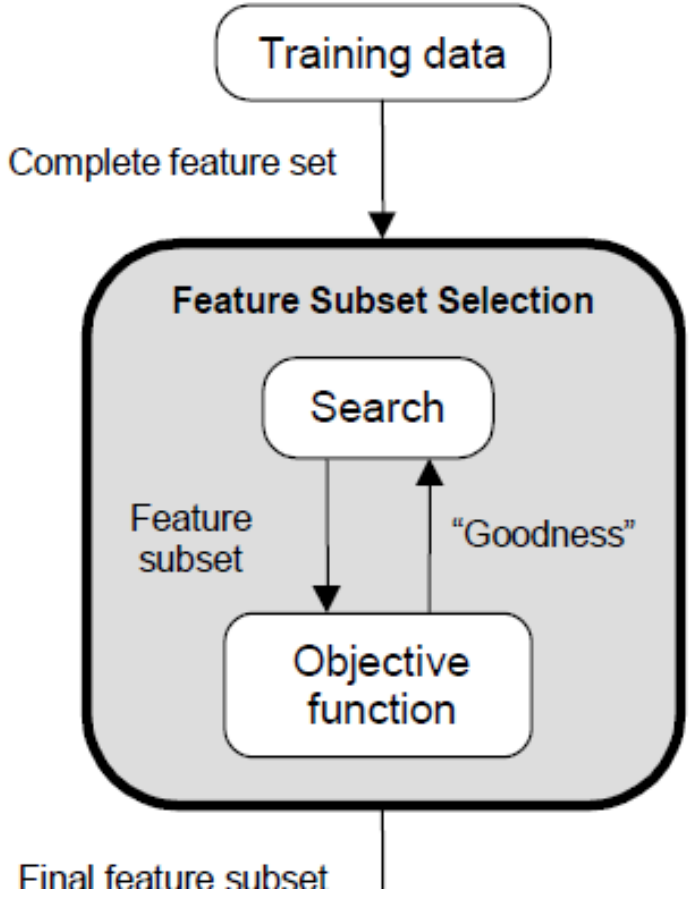
Feature Selection

- It is a process that reduces the high dimensionality of large-scale data by selecting a small subset of related and significant features and eliminating unrelated and redundant features in order to construct effective prediction models.

- It is of two broad categories: Supervised feature selection and unsupervised feature selection

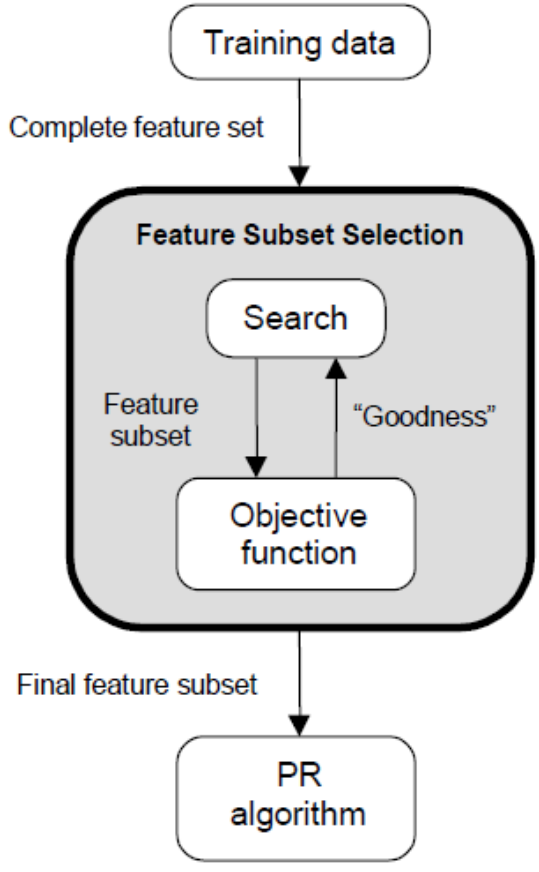
Feature Selection

- Feature selection is an **optimization** problem.
  - Search the space of possible feature subsets.
  - Pick the subset that is optimal or near-optimal with respect to some objective function.



Feature Selection

- Search methods
  - Exhaustive
  - Heuristic
- Evaluation methods
  - Filter
  - Wrapper

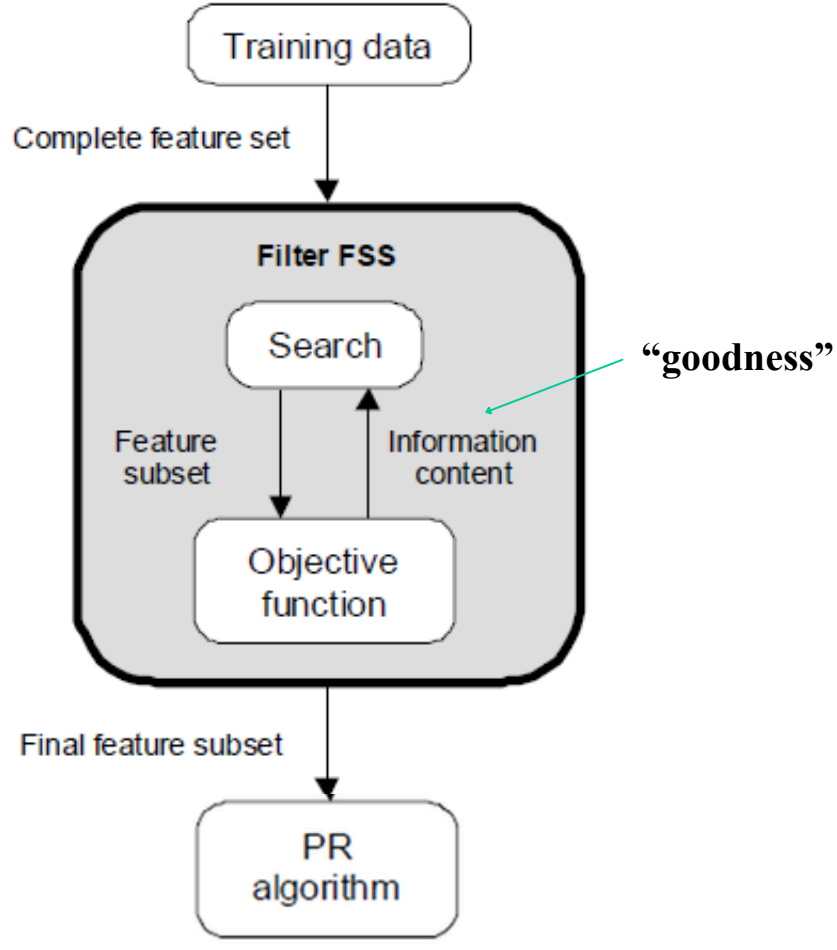


Search Methods

- Assuming n features, an exhaustive search would require examinir  possible subsets of size d.
- The number of subsets grows exponentially, and time required to find out the suitable subset of features is $O(2^n)$, which makes the exhaustive search impractical.
- In practice, heuristics are used to speed-up search but they **cannot** guarantee optimality.

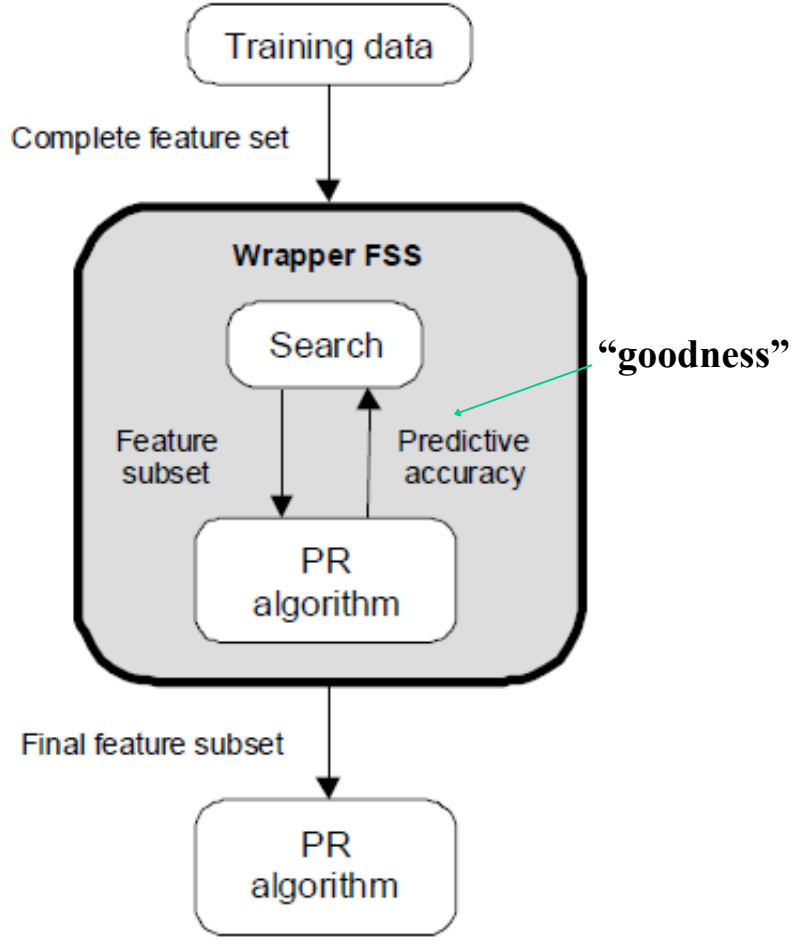
Evaluation Methods

- Filter
  - Evaluation is **independent** of the classification Algorithm.
  - The objective function is based on the information content of the feature subsets, e.g.:
    - interclass distance
    - statistical dependence
    - information-theoretic measures (e.g., mutual information).



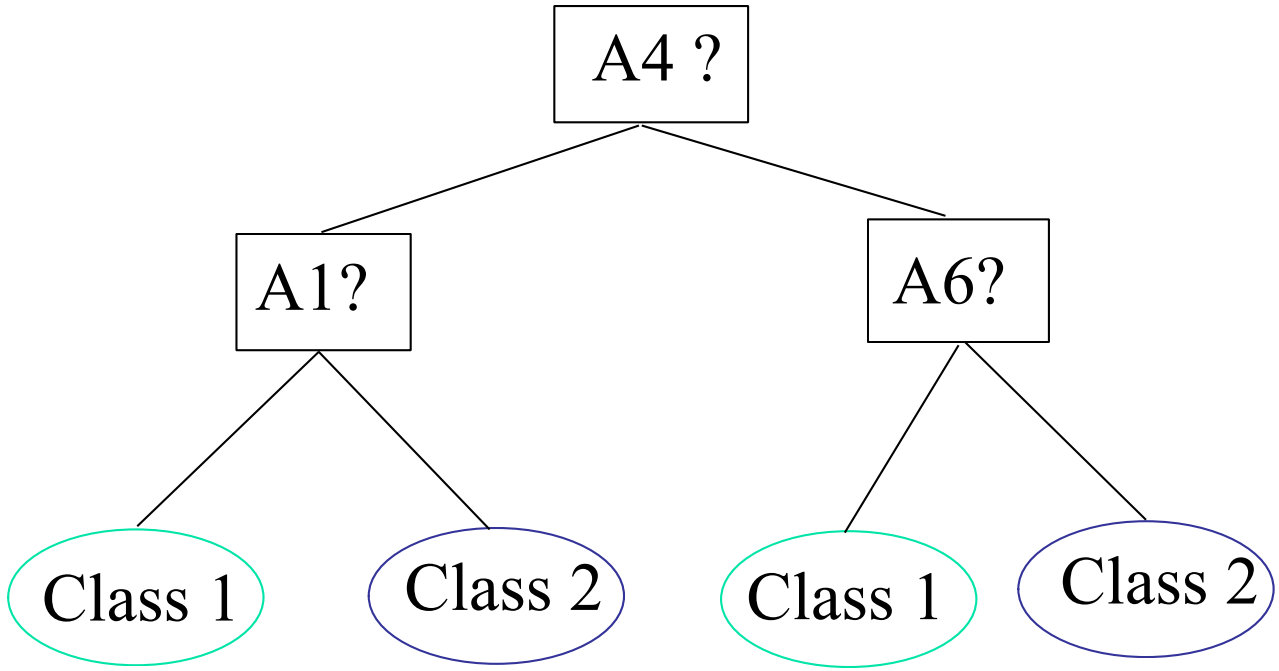
Evaluation Methods

- Wrapper
  - Evaluation uses criteria **related** to the classification algorithm.
  - The objective function is based on the predictive accuracy of the classifier



Wrapper: Decision Tree Induction

Initial attribute set:
{A1, A2, A3, A4, A5, A6}



- Selected Feature subset: {A1, A4, A6}

Filter vs Wrapper Methods

- Filter Methods
  - Advantages
    - Much faster than wrapper methods since the objective function has lower computational requirements.
    - The optimum feature set might work well with various classifiers as it is not tied to a specific classifier.
  - Disadvantages
    - Have a tendency to select more features compared to wrapper methods.
    - Achieve lower recognition accuracy compared to wrapper methods.

Filter vs Wrapper Methods

- Wrapper Methods
  - Advantages
    - Achieve higher recognition accuracy compared to filter methods since they use the classifier itself in choosing the optimum set of features.
  - Disadvantages
    - Much slower compared to filter methods since the classifier must be trained and tested for each candidate feature subset.
    - The optimum feature subset might not work well for other classifiers.

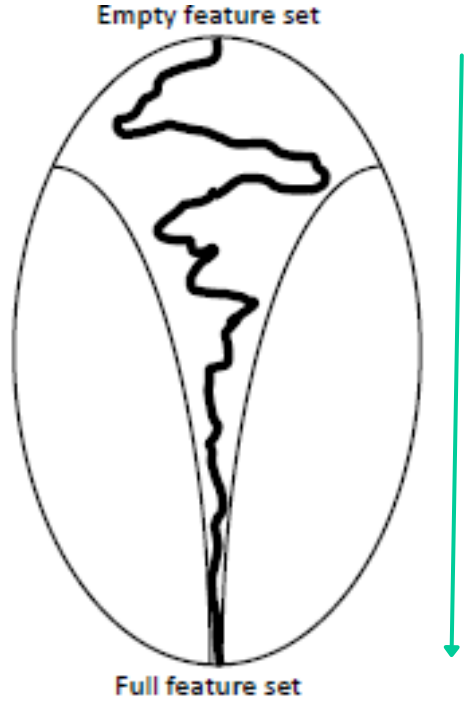
Heuristic Search - Naïve Search

- Given n features:
  - Sort them in decreasing order of their “goodness” based on some objective function.
  - Choose the top d features from this sorted list.

- Disadvantages
  - Correlation among features is not considered.
  - The best pair of features may not even contain the best individual feature.

Heuristic Search : Step-wise Forward Selection (SFS)

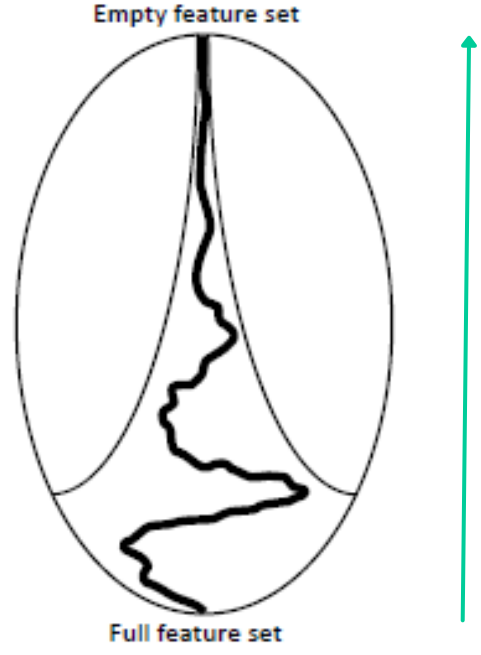
- First, the best **single** feature is selected based on some objective function.
- Then, **pairs** of features are formed using one of the remaining features and this best feature, and the best pair is selected.
- Next, **triplets** of features are formed using one of the remaining features and these two best features, and the best triplet is selected.
- This procedure continues until a predefined number of features are selected.



SFS performs best when the optimal subset is small.

Heuristic Search : Step-wise Backward Removal (SBR)

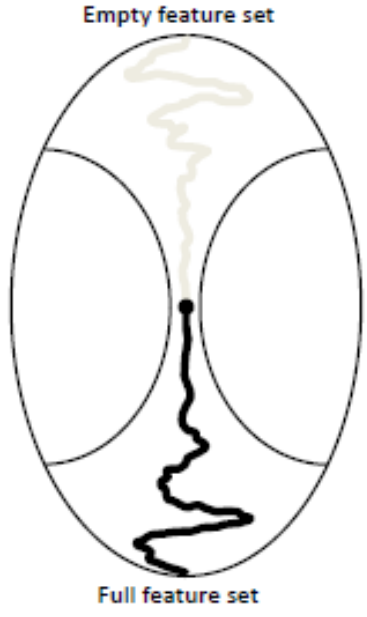
- One feature among n features is deleted:
- the objective function is computed for all subsets with **n-1** features, and the worst feature is discarded.
- Next, one feature among remaining n-1 features is deleted:
- the objective function is computed for all subsets with **n-2** features, and the worst feature is discarded.
- This procedure continues until a predefined number of features are left.



SBR performs best when the optimal subset is large.

Bidirectional Search (BDS)

- BDS applies SFS and SBR simultaneously:
- SFS is performed from the empty set.
- SBR is performed from the full set.
- To guarantee that SFS and SBR converge to the same solution:
- Features already selected by SFS cannot be removed by SBR.
- Features already removed by SBR cannot be added by SFS.



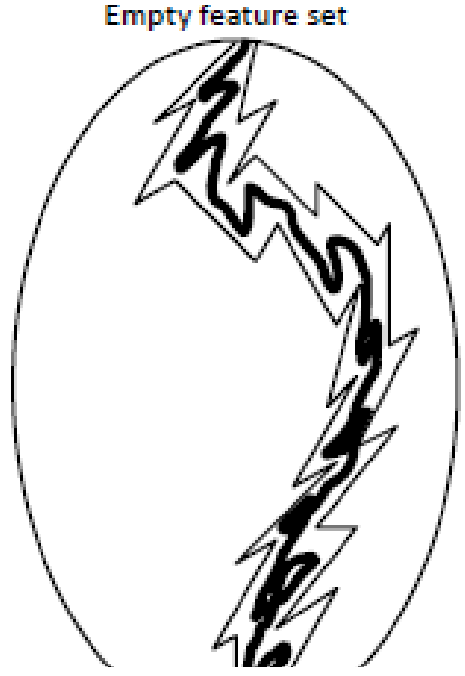
Limitations of SFS and SBR

- The main limitation of SFS is that it is unable to remove features that become non useful after the addition of other features.
- The main limitation of SBR is its inability to reevaluate the usefulness of a feature after it has been discarded.
- Enhancements of SFS and SBR:
  - "**Plus-L, minus-R**" selection (LRS)
  - **Step-wise Floating Forward Selection (SFFS)**
  - **Step-wise Floating Backward Removal (SFBR)**

"Plus-L, minus-R" selection (LRS)

(L, R are user-defined parameters)

- Let A = Feature set, F = Feature subset
- If L>R, and F is empty set:
  - Repeatedly add L features in F (by SFS)
  - Repeatedly remove R features from F into A (by SBR)
- If L<R, and F = A: (Is A = { }?)
  - Repeatedly removes R features from F (by SBR) and store them in A
  - Repeatedly add L features from A into F (by SFS)



- Main limitation - how to choose the optimal values of L and R?

Example 1: LRS

- Let A = {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9}
- Let F = { }, L=3, and R=2
- As L > R, repeat following steps until a predefined number of features are selected.
- Step 1: (i) By SFS, F={A1, A4, A8}, say; and A={A0, A2, A3, A5, A6, A7, A9}
  (ii) By SBR, F={A1}, say ; and now A ={A0, A2, A3, A4, A5, A6, A7, A8, A9}
- Step 2: (i) By SFS, F={A1, A0, A6, A9}, say; and A={A2, A3, A4, A5, A7, A8}
  (ii) By SBR, F={A1, A6}, say ; and now A ={A0, A2, A3, A4, A5, A7, A8, A9}

Is there any difference between Ex-2(i): LRS

- Let A = {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9}
- Let F = {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9}, and A={}, L=2, and R=3
- As R > L, repeat following steps until a predefined number of features are left.
- Step 1: (i) By SBR, F={A0, A2, A3, A5, A6, A7, A9} and A={A1, A4, A8}
  (ii) By SFS, F={A0, A2, A1, A3, A5, A6, A7, A8, A9}, say; and now A ={A4}
- Step 2: (i) By SBR, F={A1, A2, A3, A6, A8, A9}, say; and A={A0, A4, A5, A7}
  (ii) By SFS, F ={A1, A2, A3, A5, A6, A7, A8, A9}, say ; and now A ={A0, A4}

Is there any difference between Ex-2(ii): LRS

- Let A = {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9}
- Let F = {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9}, L=2, and R=3
- As R > L, repeat following steps until a predefined number of features are left.
- Step 1: (i) By SBR, F={A0, A2, A3, A5, A6, A7, A9} and A={A0, A1, A2, A3, A4, A5, A6, A7, A8, A9}

  (ii) By SFS, F={A0, A2, A3, A5, A6, A7, A9}, say; and now A ={A0, A1, A2, A4, A5, A6, A8, A9}
- Step 2: (i) By SBR, F={A2, A3, A6, A9}, say; and A={A0, A1, A2, A4, A5, A6, A7, A8, A9}

  (ii) By SFS, F ={A1, A2, A3, A5, A6, A9}, say ; and now A ={A0, A2, A4, A6, A7, A8, A9}

Step-wise Floating Forward Selection (SFFS) and Step-wise Floating backward Removal (SFBR)

- An extension to LRS:
  - Rather than fixing the values of L and R, floating i.e., dynamic method of determine these values from the data.
  - The size of the subset during the search can be thought to be “floating” up and down

- P. Pudil, J. Novovicova, J. Kittler, Floating search methods in feature selection, Pattern Recognition Lett. 15 (1994) 1119–1125.

Step-wise Floating Forward Selection (SFFS)

- SFFS starts from the empty set.
- After each forward step, SFFS performs multiple backward steps as long as the objective function increases.
  Example:
- Let A = {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9} and F = { }
- Step 1: By SFS (only once), F={A4}, say; and by multiple SBR, F={A4}, and A={A0, A1, A2, A3, A5, A6, A7, A8, A9}

Step-wise Floating Forward Selection (SFFS)

- Step 2: By SFS (only once), F={A4, A7}, say; and by multiple SBR, F={A4, A7}, say; and A={A0, A1, A2, A3, A5, A6, A8, A9}
- Step 3: By SFS (only once), F={A4, A7, A9}, say; and by multiple SBR, F={A4, A9}, say; and A={A0, A1, A2, A3, A5, A6, A7, A8}

- Repeat until a predefined number of features are selected

Step-wise Floating backward Removal (SFBR)

- SFBR starts from the full set.
- After each backward step, SFBR performs multiple forward steps as long as the objective function increases.

Example:

- Let A = {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9} and F = {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9}
- Step 1: By SBR (only once) from F, F={A0, A1, A2, A3, A5, A6, A7, A8, A9}, and A = {A4} say; and by multiple SFS from A, F={A0, A1, A2, A3, A5, A6, A7, A8, A9}, and A = {A4}

Step-wise Floating backward Removal (SFBR)

- Step 2: By SBR (only once) from F, F={A0, A2, A3, A5, A6, A7, A8, A9}, and A = {A1, A4} say; and by multiple SFS from A, F={A0, A2, A3, A5, A6, A7, A8, A9}, and A = {A1, A4}
- Step 3: By SBR (only once) from F, F={A0, A2, A3, A5, A6, A8, A9}, and A = {A1, A4, A7}; and by multiple SFS from A (say, no selection), F={A0, A2, A3, A5, A6, A8, A9}, and A = {A1, A4, A7}
- Step 4: By SBR (only once) from F, F={A0, A2, A3, A5, A6, A8}, and A = {A1, A4, A7, A9}; and by multiple SFS from A (say, A4 is selected), F={A0, A2, A3, A4, A5, A6, A8}, and A = {A1, A7, A9}
- Repeat until a predefined number of features are selected

Case Study on Feature Selection

- Collect various data sets using - UCI Machine Learning Repository
- Apply different data pre-processing techniques and measure the similarity between a pair of objects using different similarity and dissimilarity measurement techniques.
- Apply all Feature Selection methods discussed in the class on at least 5 data sets and compare their performance with respect to various classifiers in terms of Accuracy, Precision, Recall, and F-Score

Case Study on Feature Selection

- Collect various data sets using Datasets - UCI Machine Learning Repository
- Apply all Feature Selection methods discussed in the class on at least 5 data sets and compare their performance with respect to various classifiers in terms of Accuracy, Precision, Recall, and F-Score


Thank You